



OBSERVABILITY - RED/USE Explained for Microservices

In the world of **microservices**, **observability** is **non-negotiable**. Unlike monoliths, microservices are distributed, loosely coupled, and dynamically scaled — which makes visibility into their behavior both more complex and more critical.

To simplify observability, two powerful acronyms are used: **RED** and **USE**. Each serves a different purpose and complements the other.

What Is Observability?

Observability is not just about collecting metrics or logs; it's about the ability to **infer internal system states from the outside**.

In microservices:

- Every service can be a black box.
- Failures are partial and unpredictable.
- Dependencies (like databases, caches, queues) are external and may silently fail.

Hence, **observability** must give answers to:

- **What is broken?**
 - **Where is it broken?**
 - **Why is it broken?**
 - **Who is impacted?**
 - **How to fix it fast?**
-

RED Methodology (for Application Layer / Services)

Scope: Focused on **request-level** and **service health** metrics.

RED stands for:

Metric	Description
R - Rate	Number of requests per second handled by a service
E - Errors	Number or rate of failed requests (5xx, 4xx, exceptions)
D - Duration	Time taken to serve requests (latency, response time)

Where to Apply RED?

Use RED for:

- User-facing APIs
- Backend microservices
- Endpoints on API Gateway
- Internal service-to-service HTTP/gRPC traffic

RED in Action (Example for /checkout service)

Metric Type	Metric Name	Sample Metric
Rate	http_requests_total{service="checkout"}	4000 rps
Errors	http_requests_total{status=~"5..", service="checkout"}	0.5%
Duration	http_request_duration_seconds_bucket{le="1",service="checkout"}	p99 = 2.3s

Tools:

- Prometheus + Grafana
- AWS CloudWatch Metrics
- Datadog APM
- New Relic / Dynatrace

Alerts to Configure:

- High error rate for checkout service > 1%
- p95 latency > 3s for >5 mins
- Sudden drop in request rate (indicating crash)

USE Methodology (for Infrastructure Layer / Resources)

Scope: Focused on **resource-level** (CPU, Disk, Memory, Network) utilization of underlying systems.

USE stands for:

Metric	Description
U - Utilization	Percentage of time a resource is busy (e.g., 85% CPU)
S - Saturation	Degree to which resource has queued work (e.g., run queue length)
E - Errors	Count of error events (e.g., packet loss, disk errors, retries)

Where to Apply USE?

Use USE for:

- Containers (Docker, Podman)
- VMs / Bare Metal Nodes
- Kubernetes Nodes and Pods
- System-level debugging
- Storage systems (disks, SSDs)
- Networking (NICs, bandwidth)

USE in Action (Example for a Kubernetes Node)

Metric	Metric Name	Sample Metric
Utilization	node_cpu_utilization	87%
Saturation	node_cpu_run_queue_length	6

Errors	node_disk_errors_total	12 errors/min
--------	------------------------	---------------

Tools:

- Prometheus Node Exporter
- cAdvisor (K8s)
- Telegraf + InfluxDB
- NMON / SAR / iostat / vmstat / dstat

Alerts to Configure:

- CPU utilization > 90% + saturation > 3 for >5 mins
- Disk I/O latency spike + errors > 5/min
- Network saturation > 80% of NIC capacity

RED vs USE — Summary Table

Criteria	RED	USE
Focus	Application metrics	System/infrastructure metrics
Layer	Service/API	Container/VM/Host
Entities	Endpoints, APIs	CPU, Memory, Disk, Network
Examples	/login latency, 5xx error rate	CPU usage, Disk IO errors
Target Audience	Devs, App SREs	Infra SREs, Ops
Visualized In	API dashboards, SLO panels	Node health, resource dashboards
Alerts	High 5xx, High p95 latency	High CPU + Run queue, Disk errors

💡 How RED/USE Boosts Microservices Observability

Problem	RED Helps	USE Helps
Slow response	Identifies which endpoint is slow	Running at or near 100% utilization continuously
High failure rate	Detects failing API	Pinpoints node/memory/DNS errors
Request drop	Tracks request volume trend	Identifies saturation or OOM kills
Crash diagnosis	Which service failed?	Why did it fail (disk full, OOM)?
Autoscaling	Request rate patterns	CPU/mem usage patterns

📊 Combined Dashboards — Full Visibility

Example: Prometheus + Grafana Microservices Dashboard

App Layer (RED):

- /order Rate, p95 Latency, 4xx/5xx Error %
- Service A → Service B success vs timeout
- Correlate latency spikes with deployments

Infra Layer (USE):

- Node CPU usage + Run Queue + Load Average
- Pod memory usage vs memory limit
- Disk throughput, saturation, IO wait
- Network packet drops or retransmits

✂ Real-World Scenario

🔥 Incident: Checkout latency spike during flash sale

1. RED reveals:

- /checkout p99 latency = 5s (was 1s)
- Error rate 2%
- Spike in request rate

2. USE reveals:

- CPU utilization = 95%
- Run queue = 12 (saturation)
- Disk I/O wait = 30ms (vs <2ms)

3. Root Cause:

- CPU-starved service pods on under-provisioned nodes
- Increased garbage collection times
- Slow DB calls due to disk latency

Remediation:

- Add more pods
- Use node selectors to spread load
- Tune JVM GC
- Investigate disk I/O slowness

🚫 Best Practices

- Use RED for **Service-Level Objectives (SLOs)** and SLIs
- Use USE for **Infrastructure Health Checks**
- Always **correlate RED + USE** in alerting and dashboards
- Enable **tracing (OpenTelemetry)** for distributed request flows
- Combine with **Logs** for context (ELK, Loki, etc.)

- Integrate into **CI/CD** for canary & blue/green rollout monitoring
-

Final Thoughts

 RED + USE are **foundational pillars** for building mature observability pipelines in microservices.

They:

- Enable **fast RCA (Root Cause Analysis)**
- Improve **MTTR (Mean Time To Resolve)**
- Enhance **user experience**
- Empower **DevOps and SREs** with actionable insights

 Always remember:

- **RED = Are my services healthy?**
 - **USE = Are my systems healthy?**
-